桂林电子科技大学 2026-2027 学年 第 1 学期

数据结构与算法 实验报告

任课老师：

实验名称	**实验三** 城市间交通规划与导航		
院　　系	**计算机与信息安全学院**	专业	**计算机科学与技术（卓越）**
学　　号	**2500730126**	姓名	**王博丰**
实验日期	**2026 年 11 月 1 日**		

一、实验目的

（1）掌握图论数据结构（邻接矩阵、邻接表）在路径规划中的应用；

（2）理解最短路径算法（Dijkstra）的原理与适用场景；

（3）培养将实际问题抽象为图模型的能力，提升算法优化意识。

二、实验内容

本实验完成城市间交通规划与导航中的最省钱出行方案自动生成系统。将城市抽象为节点、道路抽象为有向边、通行费用抽象为边权，构造有向带权图。两个城市之间可能同时存在公路、铁路、航空三种线路，且三种方式的权值各不相同，一次出行可以混搭三种方式。程序读入三种方式各自的权值，用 Dijkstra 算法求起点到终点的最小费用路径，并输出每一段所采用的交通方式。

程序要求支持至少 20 个城市、50 条道路，采用命令行交互：先输入城市个数，再依次输入公路、铁路、航空的边权（每行“点 点 权值”，以 0 0 0 结束），然后输入城市与节点编号的对应关系，最后循环输入起点和终点，输入 @ @ 退出。除费用外，程序还支持通过命令行参数把权值解释为时间或距离，分别得到时间最短和距离最短的方案。对编号越界、权值非正、起终点相同、城市重名、不可达等情况，程序均给出提示。

程序设计与实现：

1、数据结构与算法

1.1 有向带权图的三层邻接矩阵

每种交通方式用一张 n×n 矩阵 Wm 表示，Wm[i][j] 是从城市 i 到城市 j 采用方式 m 的权值，0 表示该方式没有直达线路。三种方式共三张矩阵，程序用三维 vector 按 [mode][from][to] 组织。选择邻接矩阵而不是邻接表，主要有三点原因：Dijkstra 每轮松弛都要查询 u 到所有未访问 v 的三种方式权值，矩阵可以 O(1) 定位；20 个城市时三张矩阵仅 3×20×20 个整数，空间开销很小；三种方式合并到同一个邻接结构中，

矩阵比邻接表更直观，也不需要在每次松弛时遍历出边再匹配方式。

交通方式用 enum class TransportMode { ROAD, RAILWAY, AIR } 表示，既避免 0、1、2 这类魔法数字，也方便在输出时把枚举值转换成“公路”“铁路”“航空”。图中每个位置只保存一个权值，具体代表费用、时间还是距离，由运行参数决定。

```
enum class TransportMode { ROAD = 0, RAILWAY = 1, AIR = 2 };

class TransportGraph {
    int cityCount_;
    // weights_[mode][from][to]，0 表示该方式没有直达线路
    std::vector<std::vector<std::vector<int>>> weights_;
    std::vector<std::string> names_;             // 编号 -> 城市名
    std::unordered_map<std::string, int> nameToId_; // 城市名 -> 编号
};
```

1.2 城市信息的双向映射

建图时边使用 0 到 n-1 的节点编号，而查询时用户输入城市名，因此需要双向转换。程序用 unordered_map<string,int> 保存名称到编号的映射，用 vector<string> 保存编号到名称的映射。平均情况下一次查找为 O(1)，避免在每次查询时线性扫描城市数组。setCity 在写入前检查编号范围和名称是否重复，保证两个映射始终一致。

1.3 前驱信息与路径栈

经典 Dijkstra 只用一个 int 数组记录前驱，但本题还要输出每一段走的是公路、铁路还是航空，因此定义 PreInfo 结构体，同时保存前驱节点编号 preVex 和从 preVex 到当前节点所用的 transportMode。求完最短路后，从终点沿 preVex 回溯到起点，回溯过程中把 PreInfo 压入 stack；再依次出栈，就得到从起点到终点的正向路径。这样可以复用指导书给出的“栈逆转路径”思路，而不必额外保存后继指针。

出栈时有一个细节：栈元素只保存了前驱和方式，没有保存边的终点。由于栈中相邻元素存在前驱关系，当前元素对应边的终点就是下一个栈元素的 preVex；最后一段没有下一个元素，其终点就是查询终点。这样只用一个 PreInfo 数组和—个栈即可完整还原路径。

```
struct PreInfo {
    int preVex = -1;               // 前驱节点编号
    TransportMode transportMode = TransportMode::ROAD;
};

std::vector<PreInfo> preInfo(n);      // 前驱数组
std::stack<PreInfo> path;            // 回溯压栈，出栈得到正向路径
```

1.4 修改版 Dijkstra

与经典 Dijkstra 相比，本实现有三处修改，与指导书一致：第一，松弛邻居时不只看一张权值矩阵，而是遍历公路、铁路、航空三种方式，任一方式给出更小的总权值就更新；第二，前驱数组由 int 改为 PreInfo，

记录前驱和本段交通方式；第三，用栈逆转前驱链，得到正向路径。核心松弛循环如下（略去无穷判断和终点提前结束）。

```
for (int iter = 0; iter < n; ++iter) {
    int u = -1;                      // 选未访问且 dist 最小的节点
    for (int i = 0; i < n; ++i)
        if (!visited[i] && (u == -1 || dist[i] < dist[u])) u = i;
    if (u == -1) break;

    visited[u] = 1;
    for (int v = 0; v < n; ++v) {
        if (visited[v]) continue;
        for (TransportMode m : modes) {    // 三种方式逐一比较
            int w = graph.edgeWeight(m, u, v);
            if (w > 0 && dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
                preInfo[v] = {u, m};
            }
        }
    }
}
```

可以把三种方式看成同一对城市之间的三条平行边，于是原问题等价于在一张“每个城市一个节点、每对城市至多三条平行边”的图上求单源最短路。所有边权为正，Dijkstra 每轮选出的未访问节点距离已经最终确定，因此三种方式的松弛顺序不会影响结果。同一对城市同一种方式若输入多次，后输入的权值覆盖先输入的权值，这一点在建图时处理。

1.5 正确性与复杂度

设城市数为 V。每轮选最小节点 O(V)，松弛时对每个 v 检查三种方式 O(3V)，共 V 轮，所以单次查询的时间复杂度为 $O(V^2)$，空间为三张矩阵 $O(3V^2)$，另有距离、访问标记、前驱数组和路径栈各 O(V)。V=20 时三张矩阵只有 1200 个整数，单次查询约 1200 次基本操作，远低于题目规模要求。若城市数增大到上万且道路稀疏，可以改用邻接表加手写最小堆，把复杂度降到 O((V+E) log V)，但本实验的规模下矩阵实现更简单、缓存更友好。

1.6 主要数据结构汇总

表 1　主要数据结构及其作用

数据结构	定义	作用
TransportMode 枚举	ROAD / RAILWAY / AIR	标识三种交通方式，代替数字编码
weights_ 三层数组	vector[mode][from][to]	三种方式各自的邻接矩阵，0 表示无直达
nameToId_ / names_	unordered_map / vector	城市名与节点编号双向映射
PreInfo	preVex + transportMode	记录最短路前驱及本段交通方式
stack<PreInfo>	后进先出栈	逆转前驱链，出栈得到正向路径
PathResult	可达标志 + 总权值 + 节点/方式序列	保存一次查询的完整结果

2、详细设计与实现

（1）总体模块与调用关系

程序分为三个模块：graph.h/graph.cpp 负责图存储、城市双向映射和边权查询；dijkstra.h/dijkstra.cpp 负责修改版 Dijkstra 和路径还原；main.cpp 负责命令行输入、校验、查询循环与结果格式化。main 先读城市数并创建 TransportGraph，再依次读入公路、铁路、航空的边，写入对应矩阵，随后读入城市映射；每次查询调用 shortestPath，得到 PathResult 后按“起点通过方式到达……终点”的格式输出。图算法模块不直接接触输入输出，便于单独测试。模块与数据流如图 1 所示。

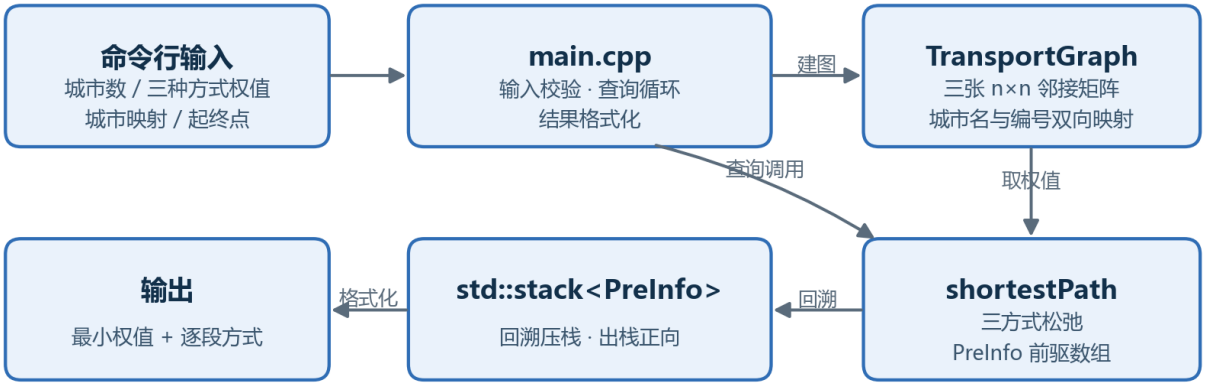


图 1　程序模块与数据流

（2）输入与校验模块

所有输入按行读取，再用 istringstream 分词，避免 operator>> 在格式错误时把残余字符留在缓冲区。城市个数限制在 1 到 1000；边输入要求一次读入三个整数，0 0 0 表示该方式输入结束；节点编号必须落在 [0, V-1]，起终点不能相同，权值必须为正整数。城市映射要求输入 V 行“城市 节点编号”，并检查编号范围、编号重复和名称重复。查询行要求两个城市，输入 @ @ 退出；找不到城市时只提示并继续，不会终止程序。输入流结束（EOF）时程序正常退出。

这些校验保证了后续算法不会访问越界下标，也避免非法输入造成死循环。把解析和校验集中在 main.cpp，图结构只接收已经合法的数据，职责更清晰。

（3）建图与城市映射模块

TransportGraph 的构造函数按城市数分配三张 n×n 矩阵并全部置 0；setEdge 写入 weights_[mode][from][to]，如果该位置原来为 0，就把边计数加一，便于统计。setCity 完成名称和编号的双向写入。三种交通方式共用同一套接口，避免为公路、铁路、航空分别写重复的存储代码。

（4）最短路径求解模块

shortestPath 先初始化距离数组为无穷、访问标记为假、前驱为 (-1, ROAD)。每轮扫描未访问节点，选距离最小的 u，标记为已访问；若 u 是终点则提前结束，因为 Dijkstra 保证此时 dist[end] 已经最终确定。然后对每个未访问的 v 和每种方式 m 取 w = edgeWeight(m, u, v)，当 w > 0 且 dist[u] + w < dist[v] 时更新距

离和前驱。距离使用 long long 保存，无穷取 long long 上限的四分之一，避免大权值累加溢出。

（5）路径还原与输出模块

若终点可达，从终点沿 preInfo 回溯并压栈，直到起点；回溯步数超过 V 时视为前驱链异常，直接返回不可达，防止死循环。随后从起点开始出栈：每个 PreInfo 给出本段方式，下一节点由下一个栈元素的 preVex 或查询终点确定。PathResult 同时保存节点序列和方式序列，输出模块只负责格式化。

```
std::stack<PreInfo> path;
for (int v = end; v != start; v = preInfo[v].preVex)
    path.push(preInfo[v]);

result.nodes.push_back(start);
while (!path.empty()) {
    PreInfo edge = path.top(); path.pop();
    int next = path.empty() ? end : path.top().preVex;
    result.modes.push_back(edge.transportMode);
    result.nodes.push_back(next);
}
```

输出格式与指导书截图一致：先输出“起点到终点的最短距离为 X”，再输出“具体行进路线为”，最后把“起点通过方式到达”逐段拼接，末尾接终点名。若起点等于终点，距离为 0，只输出城市名；若不可达，输出“起点到终点不可达”。

（6）规划依据的选择

图只保存一个权值，程序用命令行参数决定它的含义：默认不传参数时按费用，传 time 或 distance 时分别按时间和距离。三种依据共用同一套 Dijkstra 代码，只是输入提示和结果措辞不同。这样既满足了费用最少、时间最短、距离最短三种方案，又保持指导书截图中“城市数 → 三种方式权值 → 城市映射 → 查询”的输入顺序不变。如果改用交互式菜单询问规划依据，就会多读一行输入，导致严格按指导书顺序录入时第一条边被误当成菜单选项，因此最终选择了命令行参数。

3、测试方案与结果

（1）测试环境与方法

测试环境为 Windows 11（Build 26200），使用 LLVM-MinGW 的 g++/clang++ 23.1.4，按 C++17 标准并开启 -O2、-Wall、-Wextra、-Wpedantic 编译，编译退出码为 0，无警告。测试分三部分：指导书 6 城市样例、20 城市规模测试、随机图交叉验证；另外用越界、负权、自环、重名、不可达、起终点相同等输入检查错误处理，并用 AddressSanitizer 与 UBSan 运行样例检查内存和未定义行为。

（2）样例测试

按指导书 6 城市图录入公路 11 条、铁路 12 条、航空 11 条有向边，城市 v0 到 v5 对应节点 0 到 5。三组查询的运行结果如图 2 和表 2 所示。v0→v5 选择航空 v0→v1（1）、铁路 v1→v4（2）、公路 v4→v5（3），合计 6；v0→v2 为航空 v0→v1（1）加公路 v1→v2（24），合计 25；v1→v3 为铁路 v1→v4（2）、公

路 v4→v5（3）、公路 v5→v3（8），合计 13。三组结果与指导书运行截图完全一致。

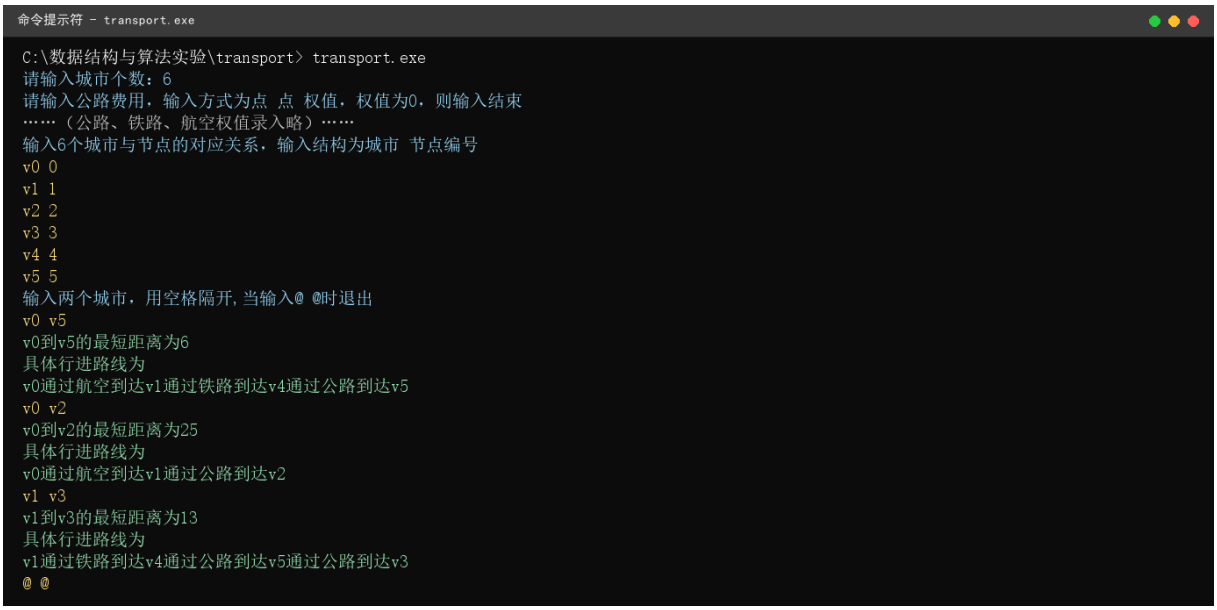


图 2　指导书 6 城市样例的查询结果

表 2　指导书样例的三组查询结果

查询	最小费用	行进路线	方式序列
v0 → v5	6	v0 → v1 → v4 → v5	航空、铁路、公路
v0 → v2	25	v0 → v1 → v2	航空、公路
v1 → v3	13	v1 → v4 → v5 → v3	铁路、公路、公路

（3）规模测试

为了验证题目要求的 20 个城市、50 条道路规模，另生成一张包含 20 个城市、74 条有向道路的测试图（公路 44 条、铁路 24 条、航空 6 条）。程序正常完成建图和查询，部分结果如图 3：北京→哈尔滨 24，上海→昆明 130，广州→西安 105，哈尔滨→北京 11。其中上海→昆明的最短路线经过 6 个城市，说明多段混搭和前驱链在较大规模下仍然正确。

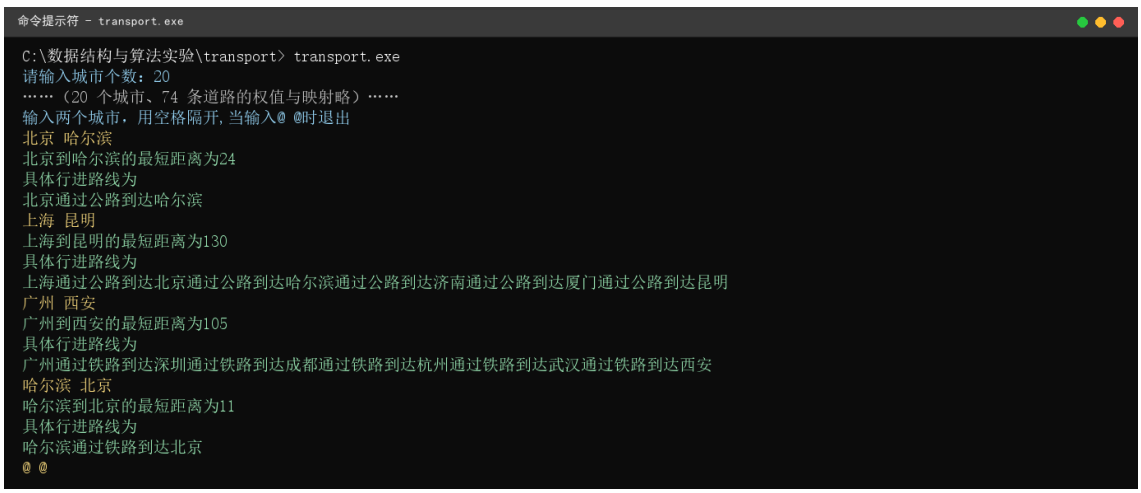


图 3　20 城市规模测试的部分结果

（4）随机交叉验证与边界情况

样例和规模测试只能说明程序在给定数据上正确，不能排除“碰巧正确”。为此另写了一个独立的 Python Dijkstra 作为参考实现，随机生成 60 张 5 到 8 个城市的图，稠密度分别取 0.15、0.3、0.5，规划依据轮流取费用、时间、距离。对每一对城市比较总权值，并逐段检查 C++ 程序输出的路线：每一条边必须真实存在，交通方式必须与矩阵一致，各段权值之和必须等于输出结果。60 张图共 1908 条可达查询全部与参考实现一致，不可达查询也全部一致。各类边界输入均按预期提示或输出。测试汇总见表 3。

表 3　测试方案与结果汇总

测试项	规模	结果
指导书样例	6 城市 / 34 条有向边	3 组查询与指导书截图一致
规模测试	20 城市 / 74 条有向边	4 组查询正常，最长路线经过 6 个城市
随机交叉验证	60 张随机图 / 1908 条可达查询	总权值与路线逐段校验全部一致
边界测试	越界、负权、自环、重名、不可达、同城	均按预期提示或输出
内存检查	AddressSanitizer + UBSan	样例运行无报错

（5）结果分析

从样例结果看，最短路径并不总是飞机最快或最便宜。v0→v5 的最优方案先用航空完成长距离段，再换铁路和公路完成最后两段，说明三种方式是互补的。程序在每次松弛时对三种方式逐一比较，能够自动选出当前最合适的方式，而不是事先假定某种方式最优。

矩阵实现的时间复杂度为 $O(V^2)$，在 V=20 时几乎没有开销；空间为 $O(3V^2)$，对题目规模完全够用。它的局限是当城市数很大且道路稀疏时，大量矩阵元素为 0，空间和时间都有浪费，此时应改为邻接表加最小堆。另一个局限是同一对城市同一方式只保存一条边，若实际中存在不同班次、不同票价的多条平行边，需要额外扩展。

三、实验小结

实验中遇到的第一个问题是如何处理三种交通方式的混搭。最初的想法是分别对公路、铁路、航空求最短路，再比较三个结果，但这样会漏掉“前半段坐飞机、后半段坐火车”这类方案。后来把三种方式看成同一对城市之间的三条平行边，在 Dijkstra 松弛时同时比较三种权值，问题就转化为普通的单源最短路；前驱结构再额外记录本段方式，就能输出完整方案。

第二个问题是路径逆转时如何确定每段边的终点。PreInfo 只保存前驱和交通方式，没有保存后继。出栈时利用栈中相邻元素的前驱关系：当前元素对应边的终点是下一个栈元素的 preVex，最后一段的终点就是查询终点。这样既保持了指导书要求的栈结构，又不用额外增加字段。

第三个问题是输入格式的兼容性。为了严格按照指导书的截图顺序录入数据，程序不能在边输入之前插入额外的交互菜单。最终把费用、时间、距离的选择放到命令行参数中，默认按费用运行，既复现了指导书样例，又保留了三种规划依据。

此外，Windows 控制台默认编码与 UTF-8 源码不一致时中文会乱码，程序在启动时调用 SetConsoleOutputCP(CP_UTF8) 和 SetConsoleCP(CP_UTF8)，编译时显式指定输入/执行字符集为 UTF-8。输入校验、EOF 处理和不可达判断也都是调试过程中补齐的，避免异常输入导致程序崩溃或死循环。

通过这次实验，我把邻接矩阵、枚举、结构体数组和栈组合成了一个完整的最短路径应用，也体会到数据结构的选择要结合具体操作：本题频繁查询任意两点三种方式的权值，矩阵比邻接表更直接；而路径逆转需要后进先出，栈正好合适。后续如果继续扩展，可以考虑邻接表加最小堆、支持多班次平行边，以及在地图上可视化路线。